

Dropshipping Architecture Selection and Justification

Software Engineering Lab 3 | Problem Statement 38

The system receives Shopify and WooCommerce order webhooks, verifies supplier stock, splits orders by vendor, sends supplier purchase orders, and records tracking numbers. Its main design constraints are prompt webhook acknowledgement, safe handling of customer delivery data, supplier failures or stock changes, and 99.5% monthly ingestion and dispatch availability.

Architectural style comparison

Style	Fit for this scenario	Main tradeoff
Layered	Clear ingress, orchestration, integration and data boundaries; supplier adapters can change independently.	A shared application needs disciplined modules and background work so slow suppliers do not block webhook acknowledgement.
Microservices	Can scale and isolate supplier integrations separately.	Network calls, distributed state and operations are costly for this five-component lab scope.
Client Server	Central server offers simple deployment and shared order state.	A central bottleneck or outage affects all stores and supplier work.

Architecture selection and two reasons

Selection. We chose Layered Architecture for the Dropshipping Inventory and Order Orchestrator System.

Reason 1. A webhook gateway, application orchestrator, supplier connector and repository have distinct responsibilities. Supplier-specific APIs can be changed behind one interface without changing the order workflow.

Reason 2. Central orchestration keeps vendor splits, stock checks, purchase-order dispatch and tracking tied to one order record. Idempotency keys prevent repeated webhooks from creating duplicate supplier purchase orders.

Security advantage

The ingress boundary verifies webhook signatures before accepting orders. The integration layer holds supplier API credentials, while the repository restricts and encrypts customer delivery details. Only the orchestrator can request purchase-order dispatch, reducing exposure across components.

Performance benefit

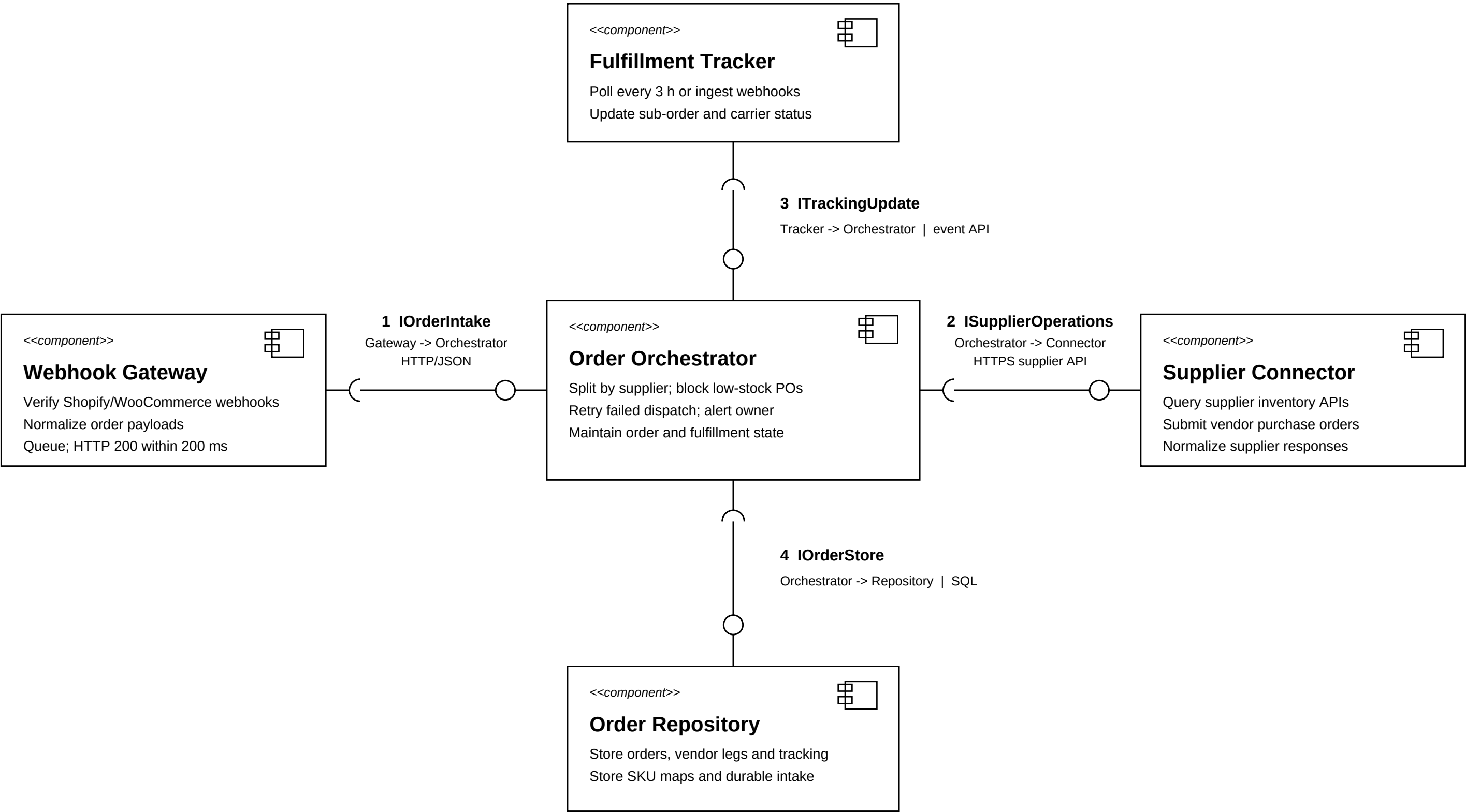
The gateway acknowledges an authenticated webhook with HTTP 200 after durable, idempotent acceptance; supplier stock checks, order splitting and purchase-order calls run afterward. This keeps slow supplier APIs off the acknowledgement path. The required acceptance target is within 200 ms under simulated peak load and must be verified by benchmarking.

Requirements coverage and reliability

The diagram has exactly five components and four internal interfaces: IOrderIntake, ISupplierOperations, ITrackingUpdate and IOrderStore. The tracker polls every three hours or ingests supplier webhooks, then updates the matching sub-order within four hours. Low stock blocks only the affected purchase order while other suppliers proceed. After two failed dispatch retries, the orchestrator sends a dashboard and email alert to the E-Store Owner within five minutes. Durable intake, idempotent records, health checks and a standby deployment support the 99.5% monthly availability target; uptime logs must verify it.

Lab 3 Component Diagram

Dropshipping Inventory and Order Orchestrator | Layered architecture | Problem statement 38



Flow	Webhook -> durable order intake -> stock check -> vendor-specific POs -> tracking update
Exceptions	Low stock blocks the affected PO; 2 failed dispatch retries trigger owner dashboard/email within 5 min.
Targets	HTTP 200 within 200 ms; tracking update within 4 h; ingestion/dispatch uptime at least 99.5% monthly.
Notation	Semicircle = required interface (caller); circle = provided interface (service). Four numbered connectors.